

JOBSHEET XI LINKED LIST



Disusun oleh:

Nama : Galih Candra Kirana
No. Absen : 10
Kelas : TI 1G
NIM : 254107020080

POLITEKNIK NEGERI MALANG
Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141
TAHUN 2025-2026

11.1 Tujuan Praktikum

Setelah melakukan materi praktikum ini, mahasiswa mampu:

1. Membuat struktur data linked list
2. Membuat linked list pada program
3. Membedakan permasalahan apa yang dapat diselesaikan menggunakan linked list

11.2 Percobaan 1: Pembuatan Single Linked List

11.2.1 Langkah-langkah Percobaan

Class Mahasiswa10:

```
package Jobsheet11.Pertemuan12;

public class Mahasiswa10 {

    String nim;

    String nama;

    String kelas;

    double ipk;

    public Mahasiswa10() {

    }

    public Mahasiswa10(String nm, String nama, String kls, double ip) {

        this.nim = nm;

        this.nama = nama;

        this.kelas = kls;

        this.ipk = ip;

    }

    void tampilInformasi() {
```

```
        System.out.println(nim + "\t\t" + nama + "\t\t" + kelas + "\t\t" + ipk);  
    }  
}
```

Class Node10:

```
package Jobsheet11.Pertemuan12;  
  
public class Node10 {  
    Mahasiswa10 data;  
    Node10 next;  
  
    public Node10(Mahasiswa10 data, Node10 next) {  
        this.data = data;  
        this.next = next;  
    }  
}
```

Class SingleLinkedList10:

```
package Jobsheet11.Pertemuan12;  
  
public class SingleLinkedList10 {  
    Node10 head;  
    Node10 tail;  
  
    boolean isEmpty() {
```

```

        return (head == null);
    }

    public void print() {
        if (!isEmpty()) {
            Node10 tmp = head;

            System.out.print("Isi Linked List:\n");

            while (tmp != null) {

                tmp.data.tampilInformasi();

                tmp = tmp.next;
            }

            System.out.println("");
        } else {

            System.out.println("Linked List kosong");
        }
    }
}

```

```

    public void addFirst(Mahasiswa10 input) {
        Node10 ndInput = new Node10(input, null);

        if (isEmpty()) {
            head = ndInput;
            tail = ndInput;
        } else {
            ndInput.next = head;
            head = ndInput;
        }
    }
}

```

```
    }  
}
```

```
public void addLast(Mahasiswa10 input) {  
    Node10 ndInput = new Node10(input, null);  
    if (isEmpty()) {  
        head = ndInput;  
        tail = ndInput;  
    } else {  
        tail.next = ndInput;  
        tail = ndInput;  
    }  
}
```

```
public void insertAfter(String key, Mahasiswa10 input) {  
    Node10 ndInput = new Node10(input, null);  
    Node10 temp = head;  
    do {  
        if (temp.data.nama.equalsIgnoreCase(key)) {  
            ndInput.next = temp.next;  
            temp.next = ndInput;  
            if (ndInput.next == null) {  
                tail = ndInput;  
            }  
            break;  
        }  
    } while (temp.next != null);  
}
```

```

    }

    temp = temp.next;
} while (temp != null);
}

public void insertAt(int index, Mahasiswa10 input) {
    if (index < 0) {
        System.out.println("Indeks salah");
    } else if (index == 0) {
        addFirst(input);
    } else {
        Node10 temp = head;

        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }

        temp.next = new Node10(input, temp.next);

        if (temp.next.next == null) {
            tail = temp.next;
        }
    }
}
}

```

Class SLLMain10:

```
package Jobsheet11.Pertemuan12;
```

```
public class SLLMain10 {  
    public static void main(String[] args) {  
        SingleLinkedList10 sll = new SingleLinkedList10();  
        Mahasiswa10 mhs1 = new Mahasiswa10("101", "Alvaro", "TI 1A", 4);  
        Mahasiswa10 mhs2 = new Mahasiswa10("102", "Bimon", "TI 3C", 3.5);  
        Mahasiswa10 mhs3 = new Mahasiswa10("103", "Cintia", "TI 2B", 3.8);  
        Mahasiswa10 mhs4 = new Mahasiswa10("104", "Dirga", "TI 4D", 3.6);  
  
        sll.print();  
        sll.addFirst(mhs4);  
        sll.print();  
        sll.addLast(mhs1);  
        sll.print();  
        sll.insertAfter("Dirga", mhs3);  
        sll.insertAt(2, mhs2);  
        sll.print();  
    }  
}
```

11.2.2 Verifikasi Hasil Percobaan

```
Linked List kosong
Isi Linked List:
104          Dirga          TI 4D          3.6

Isi Linked List:
104          Dirga          TI 4D          3.6
101          Alvaro        TI 1A          4.0

Isi Linked List:
104          Dirga          TI 4D          3.6
103          Cintia        TI 2B          3.8
102          Bimon         TI 3C          3.5
101          Alvaro        TI 1A          4.0
```

11.2.3 Pertanyaan!

1. Mengapa hasil compile kode program di baris pertama menghasilkan “Linked List Kosong”?

Jawaban:

Karena setelah di deklarasi, linked list belum memiliki node. Akibatnya, saat dilakukan pengecekan isEmpty, kondisi bernilai true sehingga menampilkan linked list kosong.

2. Jelaskan kegunaan variable temp secara umum pada setiap method!

Jawaban:

Kegunaan variabel temp pada setiap method adalah untuk menyimpan referensi node saat melakukan manipulasi linked list.

3. Lakukan modifikasi agar data dapat ditambahkan dari keyboard!

```
=== Menu ===
1. Tambahkan dari depan
2. Tambahkan dari belakang
3. Tambahkan setelah node
4. Tambahkan pada indeks
5. Cetak data
0. Keluar
Masukkan Pilihan (0-5): 5
Linked List kosong

=== Menu ===
1. Tambahkan dari depan
2. Tambahkan dari belakang
3. Tambahkan setelah node
4. Tambahkan pada indeks
5. Cetak data
0. Keluar
Masukkan Pilihan (0-5): 1
=== Data Mahasiswa ===
Masukkan NIM: 101
Masukkan nama: Dirga
Masukkan kelas: TI 4D
Masukkan ipk: 3,6

=== Menu ===
1. Tambahkan dari depan
2. Tambahkan dari belakang
3. Tambahkan setelah node
4. Tambahkan pada indeks
5. Cetak data
0. Keluar
Masukkan Pilihan (0-5): 5
Isi Linked List:
101          Dirga          TI 4D          3.6
```

```
=== Menu ===
1. Tambahkan dari depan
2. Tambahkan dari belakang
3. Tambahkan setelah node
4. Tambahkan pada indeks
5. Cetak data
0. Keluar
Masukkan Pilihan (0-5): 2
=== Data Mahasiswa ===
Masukkan NIM: 1
Masukkan nama: Alvaro
Masukkan kelas: TI 1A
Masukkan ipk: 4,0

=== Menu ===
1. Tambahkan dari depan
2. Tambahkan dari belakang
3. Tambahkan setelah node
4. Tambahkan pada indeks
5. Cetak data
0. Keluar
Masukkan Pilihan (0-5): 5
Isi Linked List:
101          Dirga          TI 4D          3.6
1            Alvaro        TI 1A          4.0
```

```

=== Menu ===
1. Tambahkan dari depan
2. Tambahkan dari belakang
3. Tambahkan setelah node
4. Tambahkan pada indeks
5. Cetak data
0. Keluar
Masukkan Pilihan (0-5): 3
Masukkan nama mahasiswa sebelumnya: Dirga
=== Data Mahasiswa ===
Masukkan NIM: 103
Masukkan nama: Cintia
Masukkan kelas: TI 2B
Masukkan ipk: 3,8

=== Menu ===
1. Tambahkan dari depan
2. Tambahkan dari belakang
3. Tambahkan setelah node
4. Tambahkan pada indeks
5. Cetak data
0. Keluar
Masukkan Pilihan (0-5): 5
Isi Linked List:
101      Dirga      TI 4D      3,6
103      Cintia    TI 2B      3,8
1        Alvaro    TI 1A      4,0

```

```

=== Menu ===
1. Tambahkan dari depan
2. Tambahkan dari belakang
3. Tambahkan setelah node
4. Tambahkan pada indeks
5. Cetak data
0. Keluar
Masukkan Pilihan (0-5): 4
Masukkan Indeks: 2
=== Data Mahasiswa ===
Masukkan NIM: 102
Masukkan nama: Bimon
Masukkan kelas: TI 3C
Masukkan ipk: 3,5

=== Menu ===
1. Tambahkan dari depan
2. Tambahkan dari belakang
3. Tambahkan setelah node
4. Tambahkan pada indeks
5. Cetak data
0. Keluar
Masukkan Pilihan (0-5): 5
Isi Linked List:
101      Dirga      TI 4D      3,6
103      Cintia    TI 2B      3,8
102      Bimon     TI 3C      3,5
1        Alvaro    TI 1A      4,0

```

Modifikasi Class SLLMain10:

package Jobsheet11.Pertemuan12;

import java.util.Scanner;

public class SLLMain10 {

```

    static Mahasiswa10 inputData(Scanner sc) {
        System.out.println("==== Data Mahasiswa ===");
        System.out.print("Masukkan NIM: ");
        String nim = sc.nextLine();
        System.out.print("Masukkan nama: ");
        String nama = sc.nextLine();
        System.out.print("Masukkan kelas: ");
        String kelas = sc.nextLine();
        System.out.print("Masukkan ipk: ");
        double ipk = sc.nextDouble();
        sc.nextLine();

```

```

        Mahasiswa10 mhs = new Mahasiswa10(nim, nama, kelas, ipk);
        return mhs;
    }

```

```

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

```

```

        SingleLinkedList10 sll = new SingleLinkedList10();

```

```

int pil;
// Mahasiswa10 mhs1 = new Mahasiswa10("101", "Alvaro", "TI 1A", 4);
// Mahasiswa10 mhs2 = new Mahasiswa10("102", "Bimon", "TI 3C",
3.5);
// Mahasiswa10 mhs3 = new Mahasiswa10("103", "Cintia", "TI 2B",
3.8);
// Mahasiswa10 mhs4 = new Mahasiswa10("104", "Dirga", "TI 4D", 3.6);

do {

    System.out.println("=== Menu ===");
    System.out.println("1. Tambahkan dari depan");
    System.out.println("2. Tambahkan dari belakang");
    System.out.println("3. Tambahkan setelah node");
    System.out.println("4. Tambahkan pada indeks");
    System.out.println("5. Cetak data");
    System.out.println("0. Keluar");
    System.out.print("Masukkan Pilihan (0-5): ");
    pil = sc.nextInt();
    sc.nextLine();

    Mahasiswa10 mhs;
    if (pil > 0 && pil <= 5) {
        switch (pil) {
            case 1:
                mhs = inputData(sc);
                sll.addFirst(mhs);
                System.out.println();
                break;

            case 2:
                mhs = inputData(sc);
                sll.addLast(mhs);
                System.out.println();
                break;

            case 3:
                System.out.print("Masukkan nama mahasiswa sebelumnya: ");
                String key = sc.nextLine();

                mhs = inputData(sc);

                sll.insertAfter(key, mhs);
                System.out.println();
                break;

```

```

        case 4:
            System.out.print("Masukkan indeks: ");
            int idx = sc.nextInt();
            sc.nextLine();

            mhs = inputData(sc);

            sll.insertAt(idx, mhs);
            System.out.println();
            break;

        case 5:
            sll.print();
            System.out.println();
            break;

        case 0:
            System.out.println("Keluar");
            break;

        default:
            System.out.println("Pilihan tidak valid.\n");
            break;
    }
}
} while (pil != 0);

// sll.print();
// sll.addFirst(mhs4);
// sll.print();
// sll.addLast(mhs1);
// sll.print();
// sll.insertAfter("Dirga", mhs3);
// sll.insertAt(2, mhs2);
// sll.print();

sc.close();
}
}

```

11.3 Percobaan 2: Modifikasi Elemen pada Single Linked List

11.3.1 Langkah – langkah percobaan

Class SingleLinkedList10:

```
package Jobsheet11.Pertemuan12;
```

```

public class SingleLinkedList10 {
    Node10 head;
    Node10 tail;

    boolean isEmpty() {
        return (head == null);
    }

    public void print() {
        if (!isEmpty()) {
            Node10 tmp = head;
            System.out.print("Isi Linked List:\n");
            while (tmp != null) {
                tmp.data.tampilInformasi();
                tmp = tmp.next;
            }
            System.out.println("");
        } else {
            System.out.println("Linked List kosong");
        }
    }

    public void addFirst(Mahasiswa10 input) {
        Node10 ndInput = new Node10(input, null);
        if (isEmpty()) {
            head = ndInput;
            tail = ndInput;
        } else {
            ndInput.next = head;

```

```
        head = ndInput;
    }
}
```

```
public void addLast(Mahasiswa10 input) {
    Node10 ndInput = new Node10(input, null);
    if (isEmpty()) {
        head = ndInput;
        tail = ndInput;
    } else {
        tail.next = ndInput;
        tail = ndInput;
    }
}
```

```
public void insertAfter(String key, Mahasiswa10 input) {
    Node10 ndInput = new Node10(input, null);
    Node10 temp = head;
    do {
        if (temp.data.nama.equalsIgnoreCase(key)) {
            ndInput.next = temp.next;
            temp.next = ndInput;
            if (ndInput.next == null) {
                tail = ndInput;
            }
            break;
        }
        temp = temp.next;
    } while (temp != null);
}
```

```

public void insertAt(int index, Mahasiswa10 input) {
    if (index < 0) {
        System.out.println("Indeks salah");
    } else if (index == 0) {
        addFirst(input);
    } else {
        Node10 temp = head;
        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }
        temp.next = new Node10(input, temp.next);
        if (temp.next.next == null) {
            tail = temp.next;
        }
    }
}

```

```

public void getData(int index) {
    Node10 tmp = head;
    for (int i = 0; i < index; i++) {
        tmp = tmp.next;
    }
    tmp.data.tampilInformasi();
}

```

```

public int indexOf(String key) {
    Node10 tmp = head;
    int index = 0;
    while (tmp != null && !tmp.data.nama.equalsIgnoreCase(key)) {

```

```

        tmp = tmp.next;
        index++;
    }

    if (tmp == null) {
        return -1;
    } else {
        return index;
    }
}

public void removeFirst() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong, tidak dapat dihapus!");
    } else if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
    }
}

public void removeLast() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong, tidak dapat dihapus!");
    } else if (head == tail) {
        head = tail = null;
    } else {
        Node10 temp = head;
        while (temp.next != tail) {
            temp = temp.next;
        }
    }
}

```

```

    }
    temp.next = null;
    tail = temp;
}
}

```

```

public void remove(String key) {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong, tidak dapat dihapus!");
    } else {
        Node10 temp = head;
        while (temp != null) {
            if ((temp.data.nama.equalsIgnoreCase(key)) && (temp == head)) {
                this.removeFirst();
                break;
            } else if (temp.data.nama.equalsIgnoreCase(key)) {
                temp.next = temp.next.next;
                if (temp.next == null) {
                    tail = temp;
                }
                break;
            }
            temp = temp.next;
        }
    }
}

```

```

public void removeAt(int index) {
    if (index == 0) {
        removeFirst();
    }
}

```

```

    } else {
        Node10 tmp = head;
        for (int i = 0; i < index - 1; i++) {
            tmp = tmp.next;
        }
        tmp.next = tmp.next.next;
        if (tmp.next == null) {
            tail = tmp;
        }
    }
}
}

```

Class SLLMain10:

```

package Jobsheet11.Pertemuan12;

import java.util.Scanner;

public class SLLMain10 {

    // static Mahasiswa10 inputData(Scanner sc) {
    //     System.out.println("=== Data Mahasiswa ===");
    //     System.out.print("Masukkan NIM: ");
    //     String nim = sc.nextLine();
    //     System.out.print("Masukkan nama: ");
    //     String nama = sc.nextLine();
    //     System.out.print("Masukkan kelas: ");
    //     String kelas = sc.nextLine();
    //     System.out.print("Masukkan ipk: ");
    //     double ipk = sc.nextDouble();
    // }
    // }
}

```

```

//    sc.nextLine();

//    Mahasiswa10 mhs = new Mahasiswa10(nim, nama, kelas, ipk);
//    return mhs;
// }

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    SingleLinkedList10 sll = new SingleLinkedList10();

    Mahasiswa10 mhs1 = new Mahasiswa10("101", "Alvaro", "TI 1A", 4);
    Mahasiswa10 mhs2 = new Mahasiswa10("102", "Bimon", "TI 3C", 3.5);
    Mahasiswa10 mhs3 = new Mahasiswa10("103", "Cintia", "TI 2B", 3.8);
    Mahasiswa10 mhs4 = new Mahasiswa10("104", "Dirga", "TI 4D", 3.6);

    // sll.print();
    sll.addFirst(mhs4);
    // sll.print();
    sll.addLast(mhs1);
    // sll.print();
    sll.insertAfter("Dirga", mhs3);
    sll.insertAt(2, mhs2);
    // sll.print();

    System.out.println("data index 1: ");
    sll.getData(1);

    System.out.println("data mahasiswa an Bimon berada pada index : " +
sll.indexOf("bimon"));
    System.out.println();

```

```

sll.removeFirst();

sll.removeLast();

sll.print();

sll.removeAt(0);

sll.print();


// int pil;

// do {
//     System.out.println("=== Menu ===");
//     System.out.println("1. Tambahkan dari depan");
//     System.out.println("2. Tambahkan dari belakang");
//     System.out.println("3. Tambahkan setelah node");
//     System.out.println("4. Tambahkan pada indeks");
//     System.out.println("5. Cetak data");
//     System.out.println("0. Keluar");
//     System.out.print("Masukkan Pilihan (0-5): ");
//     pil = sc.nextInt();
//     sc.nextLine();


//     Mahasiswa10 mhs;
//     if (pil > 0 && pil <= 5) {
//         switch (pil) {
//             case 1:
//                 mhs = inputData(sc);
//                 sll.addFirst(mhs);
//                 System.out.println();
//                 break;


//             case 2:

```

```
        //      mhs = inputData(sc);
        //      sll.addLast(mhs);
        //      System.out.println();
        //      break;

        //      case 3:
        //      System.out.print("Masukkan nama mahasiswa sebelumnya:
");
        //      String key = sc.nextLine();

        //      mhs = inputData(sc);

        //      sll.insertAfter(key, mhs);
        //      System.out.println();
        //      break;

        //      case 4:
        //      System.out.print("Masukkan indeks: ");
        //      int idx = sc.nextInt();
        //      sc.nextLine();

        //      mhs = inputData(sc);

        //      sll.insertAt(idx, mhs);
        //      System.out.println();
        //      break;

        //      case 5:
        //      sll.print();
        //      System.out.println();
        //      break;
```

```

//      case 0:
//          System.out.println("Keluar");
//          break;

//      default:
//          System.out.println("Pilihan tidak valid.\n");
//          break;
//      }
//  }
// } while (pil != 0);

sc.close();
}
}

```

11.3.2 Verifikasi Hasil Percobaan

```

data index 1:
103          Cintia          TI 2B          3.8
data mahasiswa an Bimon berada pada index : 2

Isi Linked List:
103          Cintia          TI 2B          3.8
102          Bimon          TI 3C          3.5

Isi Linked List:
102          Bimon          TI 3C          3.5

```

11.3.3 Pertanyaan!

1. Mengapa digunakan keyword break pada fungsi remove? Jelaskan!
Jawaban:
Fungsi break pada fungsi remove adalah untuk menghentikan proses sampai sebelum break dan langsung keluar (kode setelah break tidak diproses).
2. Jelaskan kegunaan kode dibawah pada method remove

```

1 temp.next = temp.next.next;
2 if (temp.next == null) {
3     tail = temp;
4 }

```

Jawaban:

Kode tersebut berfungsi untuk mengubah referensi pointer node temp agar melewati 1 node, setelah itu dilakukan pengecekan jika referensi pointer node temp yang telah diubah tadi mengarah ke null, maka variabel tail diisi dengan node temp.

11.4 Tugas

11.4.1 Kode Program

Class Mahasiswa10:

```
package Jobsheet11.Pertemuan12.Tugas;

public class Mahasiswa10 {
    String nim;
    String nama;
    String prodi;
    String kelas;

    public Mahasiswa10(String nim, String nama, String prodi, String kelas) {
        this.nim = nim;
        this.nama = nama;
        this.prodi = prodi;
        this.kelas = kelas;
    }

    public void tampilkanData() {
        System.out.println(nim + "\t\t" + nama + "\t\t" + prodi + "\t\t" + kelas);
    }
}
```

Class Node10:

```
package Jobsheet11.Pertemuan12.Tugas;

public class Node10 {
    Mahasiswa10 data;
    Node10 next;

    public Node10(Mahasiswa10 data, Node10 next) {
        this.data = data;
        this.next = next;
    }
}
```

Class AntrianLayanan10:

```
package Jobsheet11.Pertemuan12.Tugas;

public class AntrianLayanan10 {
    Node10 head;
    Node10 tail;
    int size = 0;

    boolean isEmpty() {
        return (head == null);
    }

    public void print() {
        if (!isEmpty()) {
            Node10 tmp = head;
            System.out.println("=== Data Antrian ===");
            while (tmp != null) {
                tmp.data.tampilkanData();
                tmp = tmp.next;
            }
        } else {
            System.out.println("Antrian masih kosong.");
        }
    }

    public void clear() {
        if (head != null && tail != null) {
            head = null;
            tail = null;
            System.out.println("Antrian berhasil dikosongkan.");
        } else {
            System.out.println("Antrian masih kosong.");
        }
    }

    public void addLast(Mahasiswa10 input) {
        Node10 ndInput = new Node10(input, null);
        if (isEmpty()) {
            head = ndInput;
            tail = ndInput;
        } else {
            tail.next = ndInput;
            tail = ndInput;
        }
        size++;
    }
}
```

```

public void getData(int index) {
    Node10 tmp = head;
    for (int i = 0; i < index; i++) {
        tmp = tmp.next;
    }
    tmp.data.tampilkanData();
}

public void getDataTail() {
    Node10 tmp = tail;
    tmp.data.tampilkanData();
}

public int getJmlAntrian() {
    return size;
}

public void removeFirst() {
    if (isEmpty()) {
        System.out.println("Antrian masih kosong, tidak dapat melayani!");
    } else if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
    }
    System.out.println("Antrian berhasil dilayani.");
}
}

```

Class LayananAkademikMhs10:

```

package Jobsheet11.Pertemuan12.Tugas;
import java.util.Scanner;

public class LayananAkademikMhs10 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        AntrianLayanan10 antrian = new AntrianLayanan10();
        int pilihan;

        do {
            System.out.println("\n=== Menu Antrian Layanan Kemahasiswaan
            ===");
            System.out.println("1. Cek Antrian");

```

```

System.out.println("2. Tambah Antrian Mahasiswa");
System.out.println("3. Layanan Mahasiswa");
System.out.println("4. Lihat Mahasiswa Terdepan");
System.out.println("5. Lihat Mahasiswa Terbelakang");
System.out.println("6. Jumlah Mahasiswa dalam Antrian");
System.out.println("0. Keluar");
System.out.print("Pilih menu: ");
pilihan = sc.nextInt();
sc.nextLine();

int nextPill;

switch (pilihan) {
    case 1:
        do {
            System.out.println("1. Cek antrian kosong");
            System.out.println("2. Cek antrian penuh");
            System.out.println("3. Mengosongkan antrian");
            System.out.println("0. Kembali");
            System.out.print("Masukkan pilihan: ");
            nextPill = sc.nextInt();
            sc.nextLine();

            switch (nextPill) {
                case 1:
                    System.out.println(antrian.isEmpty() ? "Antrian
kosong.\n" : "Antrian sudah ada isinya.\n");
                    break;

                case 2:
                    System.out.println("Tidak bisa penuh karena linked list
bersifat dinamis.\n");
                    break;

                case 3:
                    antrian.clear();
                    break;

                case 0:
                    System.out.println("Kembali.");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!\n");
                    break;
            }
        }
    }
}

```

```

        } while (nextPill != 0);
        break;

    case 2:
        System.out.print("NIM    : ");
        String nim = sc.nextLine();
        System.out.print("Nama    : ");
        String nama = sc.nextLine();
        System.out.print("Prodi   : ");
        String prodi = sc.nextLine();
        System.out.print("Kelas  : ");
        String kelas = sc.nextLine();
        Mahasiswa10 mhs = new Mahasiswa10(nim, nama, prodi, kelas);
        antrian.addLast(mhs);
        break;

    case 3:
        antrian.removeFirst();
        break;

    case 4:
        antrian.getData(0);
        break;

    case 5:
        antrian.getDataTail();
        break;

    case 6:
        System.out.println("Jumlah dalam antrian: " +
antrian.getJmlAntrian());
        antrian.print();
        break;

    case 0:
        System.out.println("Terima kasih.");
        break;

    default:
        System.out.println("Pilihan tidak valid.");
    }
} while (pilihan != 0);
sc.close();
}
}

```

11.4.2 Output

```
=== Menu Antrian Layanan Kemahasiswaan ===
1. Cek Antrian
2. Tambah Antrian Mahasiswa
3. Layanan Mahasiswa
4. Lihat Mahasiswa Terdepan
5. Lihat Mahasiswa Terbelakang
6. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 1
1. Cek antrian kosong
2. Cek antrian penuh
3. Mengosongkan antrian
0. Kembali
Masukkan pilihan: 1
Antrian kosong.

1. Cek antrian kosong
2. Cek antrian penuh
3. Mengosongkan antrian
0. Kembali
Masukkan pilihan: 0
Kembali.

=== Menu Antrian Layanan Kemahasiswaan ===
1. Cek Antrian
2. Tambah Antrian Mahasiswa
3. Layanan Mahasiswa
4. Lihat Mahasiswa Terdepan
5. Lihat Mahasiswa Terbelakang
6. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 2
NIM      : 101
Nama      : Galih
Prodi     : TI
Kelas    : TI 1G
```

```
=== Menu Antrian Layanan Kemahasiswaan ===
1. Cek Antrian
2. Tambah Antrian Mahasiswa
3. Layanan Mahasiswa
4. Lihat Mahasiswa Terdepan
5. Lihat Mahasiswa Terbelakang
6. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 2
NIM      : 102
Nama      : Candra
Prodi     : SIB
Kelas    : SIB 1H

=== Menu Antrian Layanan Kemahasiswaan ===
1. Cek Antrian
2. Tambah Antrian Mahasiswa
3. Layanan Mahasiswa
4. Lihat Mahasiswa Terdepan
5. Lihat Mahasiswa Terbelakang
6. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 1
1. Cek antrian kosong
2. Cek antrian penuh
3. Mengosongkan antrian
0. Kembali
Masukkan pilihan: 1
Antrian sudah ada isinya.

1. Cek antrian kosong
2. Cek antrian penuh
3. Mengosongkan antrian
0. Kembali
Masukkan pilihan: 0
Kembali.
```

```
=== Menu Antrian Layanan Kemahasiswaan ===
1. Cek Antrian
2. Tambah Antrian Mahasiswa
3. Layanan Mahasiswa
4. Lihat Mahasiswa Terdepan
5. Lihat Mahasiswa Terbelakang
6. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 4
101      Galih      TI      TI 1G

=== Menu Antrian Layanan Kemahasiswaan ===
1. Cek Antrian
2. Tambah Antrian Mahasiswa
3. Layanan Mahasiswa
4. Lihat Mahasiswa Terdepan
5. Lihat Mahasiswa Terbelakang
6. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 5
102      Candra     SIB     SIB 1H

=== Menu Antrian Layanan Kemahasiswaan ===
1. Cek Antrian
2. Tambah Antrian Mahasiswa
3. Layanan Mahasiswa
4. Lihat Mahasiswa Terdepan
5. Lihat Mahasiswa Terbelakang
6. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 6
Jumlah dalam antrian: 2
=== Data Antrian ===
101      Galih      TI      TI 1G
102      Candra     SIB     SIB 1H
```

```
=== Menu Antrian Layanan Kemahasiswaan ===
1. Cek Antrian
2. Tambah Antrian Mahasiswa
3. Layanan Mahasiswa
4. Lihat Mahasiswa Terdepan
5. Lihat Mahasiswa Terbelakang
6. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 3
Antrian berhasil dilayani.

=== Menu Antrian Layanan Kemahasiswaan ===
1. Cek Antrian
2. Tambah Antrian Mahasiswa
3. Layanan Mahasiswa
4. Lihat Mahasiswa Terdepan
5. Lihat Mahasiswa Terbelakang
6. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 6
Jumlah dalam antrian: 2
=== Data Antrian ===
102      Candra     SIB     SIB 1H
```